

kmhelpers: A Python toolkit for automated management of genomic indexes

Sébastien Bellenous¹ and Pierre Peterlongo¹ 

¹ Genscale, Univ. Rennes, Inria, CNRS, IRISA - UMR 6074, Rennes, F-35000 France

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Large-scale genomic sequence search has become a central challenge in modern bioinformatics [REFSTODO]. A family of scalable methods uses Bloom filters (BF) (Bloom, 1970). At query time, this enables assigning the queried k -mers to the set of datasets they belong to. Building such indexes on a vast set of genomic samples requires several steps. First, determine the length of each sample (in terms of its number of distinct k -mers). Second, given a user-defined false positive rate, distribute input samples into distinct groups, each BF of a group having the same fixed size. The sizes of BF of groups have to be optimized to reduce the final whole index size. Third, given computational constraints, the whole construction is split into jobs, each generating sub-parts of the index, and a final step merges these sub-parts.

Those steps require expertise, and any error at any stage can invalidate downstream results and waste significant computation time.

$\$$ – $\#$ k -mer indexes built on Bloom filters allow efficient querying of large sequence collections. However, building and managing such indexes in practice involve a complex chain of steps.

$\$$ – $\#$ such as those produced by `kmindex` (Téo Lemané & others, 2024), : enumerating sample files, counting k -mers, selecting appropriate Bloom-filter parameters, generating index definitions, orchestrating build jobs, and compressing outputs for long-term storage. Each step requires expertise, and errors at any stage can invalidate downstream results and waste significant computation time.

We propose `kmhelpers`, an open-source Python toolkit that automates this entire workflow for building (and querying) indexes created using `kmindex` (Téo Lemané & others, 2024). `kmhelpers` provides both a command-line interface (CLI) and a Python API, covering every stage of the k -mer index lifecycle.

Statement of Need

k -mer based sequence databases built with tools like `kmindex` and `kmtricks` (Thibault Lemané et al., 2022) are used to address large-scale questions in genomics, metagenomics, and population studies. For instance, those tools were recently used to index 50 petabases of raw sequencing data (Chikhi et al., 2025). However, the gap between having the raw indexing software and running a reproducible, end-to-end optimized indexing workflow is significant, and reserved to specialists.

The work we propose in this work, `kmhelpers` aims to popularize and make accessible the process of building or update a genomic search engine from a raw set of data. This opens the doors for non-specialists to the creation and maintainability of private (such as in hospitals) or shared indexes (that can be registered in larger search engines).

39 Formally, the input is a set $\mathcal{S} = S_1, \dots, S_n$ of n genomic samples of various sizes. Each S_i can
 40 be either a raw sequencing dataset or assembled sequences. The output is an index built using
 41 kmindex, subject to two main user-defined limits: (1) the maximum allowed false-positive rate,
 42 and (2) the maximum number of sub-indexes, each being a file storing a set of BFs as a matrix.

43 One needs to briefly explain the concept of sub-index. The main challenge arises from the
 44 fact that each BF size must be adapted to the number of items it contains. In the context
 45 of this work, a BF indexes all distinct k -mers of a sample S_i . Suppose first that all samples
 46 had the same number of distinct k -mers: all BFs would then have the same size and could be
 47 merged into a single matrix. In such a matrix, a column is a BF and a row is of length n , and
 48 it represents the presence (1) or absence (0) of a k -mer across all n samples, assuming the
 49 same hash function is used for all BFs. This is the ideal situation, as only a single file needs to
 50 be accessed and opened at query time.

51 In practice, sample sizes differ from one another. One solution would be to adapt the BF size
 52 to the largest sample. However, sample sizes can vary by several orders of magnitude, which
 53 would result in enormous storage overhead. The opposite extreme would involve creating one
 54 matrix per sample (each composed of a single column). This would be optimal in terms of
 55 storage, but would dramatically slow down the query process, as n distinct files would need
 56 to be accessed for each queried k -mer. The solution we propose is a middle ground. The
 57 user defines the maximum number of sub-groups — hence the maximum number of matrices
 58 created — and, as shown in the pipeline below, kmhelpers automatically determines the BF
 59 size for each sub-index such that the total index size is minimised.

60 PIERRE: STOP ICI The identified needs follow the following steps kmhelper is a pipeline
 61 - sample discovery and k -mer counting (list), - Bloom-filter span profiling and grouping
 62 (profile), - index definition generation (compose), - upfront validation of sample files, available
 63 disk space and memory with generation of ready-to-execute pipeline scripts (plan), - index
 64 building from definition files (apply), - sequence querying (query), - ZSTD-based index (block)
 65 compression (compress) (?), - registry management (registry).

66 Multi-step workflows can be described as declarative YAML pipelines (pipeline) and executed
 67 in a single command.

68 **Parameter selection.** Bloom filter-based indexes require choosing a filter size (or *span*) per
 69 sample. Choosing this parameter incorrectly leads to either storage consumption or unacceptably
 70 high false-positive rates. kmhelpers automates this with its profile command, which reads
 71 k -mer count distributions produced by ntCard (Mohamadi et al., 2017) and assigns each
 72 sample to the smallest span that keeps the false-positive rate below a user-defined threshold.

73 State of the Field

74 Several tools address large-scale sequence search using k -mer based data structures. BIGSI
 75 (Bradley et al., 2019) and COBS (Bingmann et al., 2019) use compressed Bloom filter matrices
 76 to answer presence/absence queries across collections of sequencing experiments. HowDeSBT
 77 (Harris & Medvedev, 2020) and Mantis (Pandey et al., 2018) use sequence Bloom trees and
 78 counting quotient filters, respectively, to support similar queries. kmindex, which kmhelpers
 79 wraps, builds on the kmtricks counting pipeline and is designed for efficient querying of large
 80 sequence collections.

81 kmhelpers does not compete with any of these approaches at the algorithmic level. Its
 82 contribution is orthogonal: it provides the workflow automation, parameter optimisation,
 83 and index lifecycle management that are necessary to deploy kmindex-based databases in
 84 practice, but are absent from the core tool. Similar automation layers exist for other complex
 85 bioinformatics pipelines (e.g., Snakemake (Köster & Rahmann, 2012) workflows wrapping
 86 alignment or assembly tools), but no dedicated automation layer existed for kmindex prior to
 87 kmhelpers.

88 Design and Implementation

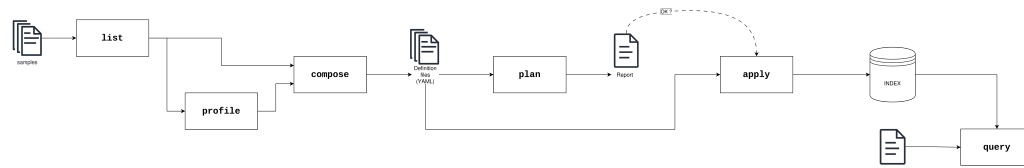


Figure 1: Overview of the kmhelpers workflow. `list` enumerates sample files and counts k -mers; `profile` analyses the count distributions to assign each sample to a Bloom-filter span and recommend index groupings. Both outputs feed into `compose`, which generates YAML index definition files. `plan` then validates sample paths, available disk space and memory, and emits a ready-to-execute pipeline script; the resulting report is reviewed before committing to the build. `apply` reads the definition files and invokes `kmindex` to construct the index. Finally, `query` searches the built index against user-provided sequences and returns ranked results.

89 [[TODO]]

90 kmhelpers is implemented in Python (≥ 3.8) and distributed via Conda with automatic
91 installation of its bioinformatics dependencies (`kmindex`, `ntCard`). The CLI is built with Click
92 (Ronacher & Click Contributors, 2023) and the package exposes a public Python API covering
93 all CLI functionality.

94 **Workflow orchestration.** Building a large index may involve hundreds to millions of samples
95 spread across multiple sub-indexes. Managing these build jobs manually is error-prone.
96 kmhelpers introduces a declarative index definition format (YAML) and two complementary
97 commands: `plan`, which validates sample files, available disk space and memory upfront and
98 emits ready-to-execute pipeline scripts; and `apply`, which reads the definition files and runs
99 the build, with support for span-level and name-level filtering.

100 Together, these features make k -mer indexing workflows accessible to researchers who are
101 not experts in the underlying data structures, while remaining flexible enough for large-scale
102 production use.

103 Acknowledgements

104 The authors thank Téo Lemane for developing `kmindex` and for his responsiveness in
105 addressing feature requests and issues raised during the development of kmhelpers. We
106 acknowledge the GenOuest core facility (<https://www.genouest.org>) for providing the
107 computing infrastructure. The work was funded by the Inria Challenge “OmicFinder”
108 (<https://project.inria.fr/omicfinder/>).

109 AI Usage Disclosure

110 AI assistance was used for constrained tasks (drafting, editing, code suggestions) under strict
111 human review at every stage. AI provider used: Claude (Anthropic, 2025).

112 References

- 113 Bingmann, T., Bradley, P., Gauger, F., & Iqbal, Z. (2019). COBS: A compact bit-sliced
114 signature index. *String Processing and Information Retrieval (SPIRE 2019)*. https://doi.org/10.1007/978-3-030-32686-9_21
115

- 116 Bloom, B. H. (1970). Space/time trade-offs in hash coding with allowable errors.
117 *Communications of the ACM*, 13(7), 422–426.
- 118 Bradley, P., Den Bakker, H. C., Rocha, E. P. C., McVean, G., & Iqbal, Z. (2019). Ultrafast
119 search of all deposited bacterial and viral genomic data. *Nature Biotechnology*, 37(2),
120 152–159. <https://doi.org/10.1038/s41587-018-0010-1>
- 121 Chikhi, R., Lemane, T., Loll-Krippléber, R., Montoliu-Nerin, M., Raffestin, B., Camargo, A. P.,
122 Miller, C. J., Fiamenghi, M. B., Agostinho, D. P., Majidian, S., & others. (2025). Logan:
123 Planetary-scale genome assembly surveys life's diversity. *bioRxiv*, 2024–2007.
- 124 Harris, R. S., & Medvedev, P. (2020). Improved representation of sequence Bloom trees.
125 *Bioinformatics*, 36(3), 721–727. <https://doi.org/10.1093/bioinformatics/btz662>
- 126 Köster, J., & Rahmann, S. (2012). Snakemake—a scalable bioinformatics workflow engine.
127 *Bioinformatics*, 28(19), 2520–2522. <https://doi.org/10.1093/bioinformatics/bts480>
- 128 Lemane, Thibault, Medvedev, P., Chikhi, R., & Peterlongo, P. (2022). Kmtricks: Efficient and
129 flexible construction of Bloom filters for large sequencing data collections. *GigaScience*, 11,
130 giac029. <https://doi.org/10.1093/gigascience/giac029>
- 131 Lemane, Téo, & others. (2024). Indexing and real-time user-friendly queries in terabyte-sized
132 complex genomic datasets with kmindex and ORA. *Nature Computational Science*, 4(2),
133 104–109. <https://doi.org/10.1101/2023.05.31.543043>
- 134 Mohamadi, H., Khan, H., & Birol, I. (2017). ntCard: A streaming algorithm for cardinality
135 estimation in genomics data. *Bioinformatics*, 33(9), 1324–1325. <https://doi.org/10.1093/bioinformatics/btw832>
- 137 Pandey, P., Almodaresi, F., Bender, M. A., Ferdman, M., Johnson, R., & Patro, R. (2018).
138 Mantis: A fast, small, and exact large-scale sequence-search index. *Cell Systems*, 7(2),
139 201–207.e4. <https://doi.org/10.1016/j.cels.2018.05.021>
- 140 Ronacher, A., & Click Contributors. (2023). *Click: Python composable command line interface*
141 *toolkit*. <https://click.palletsprojects.com>